

TEORÍA DE LA COMPUTACIÓN Y VERIFICACIÓN DE PROGRAMAS. Examen 1 Año 2022.

El examen dura 2 horas. No desarrollar nada ya hecho en clase, pero sí indicar de dónde proviene. Ser breves y claros. No dedicar mucho tiempo a ningún ejercicio. Hay que hacer ejercicios de las tres unidades. ¡Mucha suerte!

Computabilidad

Ejercicio 1. Sean un lenguaje L_1 de RE y un lenguaje L_2 de R.

1. Probar, construyendo una MT, que $L_1 \cap L_2$ es un lenguaje de RE. *Comentario:* sólo se pide presentar la idea general de la MT.
2. Encontrar un lenguaje L_1 de RE y un lenguaje L_2 de R tales que $L_1 \cap L_2$ no esté en R.

Ejercicio 2. Construir una MT que acepte el lenguaje $L = \{ \langle M \rangle, w_1, w_2 \mid \text{la MT } M \text{ acepta al menos una de las cadenas } w_i \}$. *Comentario:* sólo se pide presentar la idea general de la MT.

Ejercicio 3. La siguiente es una prueba de que L_U no está en R:

- a. Existe un lenguaje L tal que L^c no está en RE y L se reduce a L_U .
- b. Como L^c no está en RE, entonces L no está en R.
- c. Y como L no está en R y L se reduce a L_U , entonces L_U no está en R.

Se pide justificar los incisos (b) y (c). *Ayuda:* tener en cuenta las propiedades estudiadas en clase sobre las clases R y RE y las reducciones de lenguajes.

Ejercicio 4. Probar que si A es Turing-reducible a B , entonces A^c es Turing-reducible a B^c . *Ayuda:* teniendo en cuenta la hipótesis, hay que probar que existe una MT con oráculo B^c que acepta A^c y se detiene siempre.

Complejidad Computacional

Ejercicio 5. Probar que cualquier lenguaje L_1 de P se puede reducir polinomialmente a cualquier lenguaje L_2 de P, siempre que L_2 no sea Σ^* ni $\emptyset$. *Comentario:* describir la función de reducción de L_1 a L_2 y probar que la función es polinomial.

acepta A^C y se detiene siempre.

Complejidad Computacional

Ejercicio 5. Probar que cualquier lenguaje L_1 de P se puede reducir polinomialmente a cualquier lenguaje L_2 de P , siempre que L_2 no sea Σ^* ni $\emptyset$. Comentario: describir la función de reducción de L_1 a L_2 y probar que la función se computa en tiempo polinomial.

Ejercicio 6. Construir una MT que acepte todos los pares $\langle M, n \rangle$, siendo n un número natural, que cumplen que M , a partir del input vacío, se detiene en a lo sumo n^2 pasos. Ayuda: la función n^2 es tiempo-construible. Comentario: sólo presentar la idea general de la MT.

Ejercicio 7. Probar que el siguiente lenguaje está en NP: $\text{TRUE-SAT} = \{\varphi \mid \varphi \text{ es una fórmula booleana, sin cuantificadores, y satisficible con al menos dos asignaciones de valores de verdad, en una de las cuales todos los valores de verdad son true}\}$. Ayuda: la evaluación de una asignación sobre una fórmula booleana tarda tiempo polinomial.

Verificación de Programas

Ejercicio 8. Indicar si las siguientes fórmulas de correctitud son verdaderas o falsas. Justificar la respuesta en cada caso. Comentario: no hay que utilizar los métodos H y H^* , sólo aplicar las definiciones de correctitud parcial y total, y recordar que las variables son de tipo entero:

1. $\{ \text{true} \} x := x + y \{ x \geq y \}$
2. $\{ x=0 \} \text{ while } y = 0 \text{ do } z := z + 1 \text{ od } \{ x=0 \}$
3. $\langle x > 0 \wedge y > 0 \rangle \text{ while } y > x \text{ do } y := y - x \text{ od } \langle \text{true} \rangle$

Ejercicio 9. Dado un programa S del lenguaje PLW estudiado en clase, justificar en cada caso por qué son verdaderos los siguientes enunciados:

1. Que exista algún estado σ tal que $\sigma \models p$ y $\text{val}(\pi(S, \sigma)) \models q$, no implica que $\models \{p\} S \{q\}$.
2. Que exista algún estado σ tal que $\sigma \not\models p$ y $\text{val}(\pi(S, \sigma)) \not\models q$, no implica que $\models \langle p \rangle S \langle q \rangle$.

Ejercicio 10. Probar con el método H la fórmula $\{p\} \text{ while } p \text{ do } S \text{ od } \{q\}$, asumiendo que en H se prueba la fórmula $\{p\} S \{p\}$. Ayuda: Hay que probar con H la fórmula $\{p\} \text{ while } p \text{ do } S \text{ od } \{q\}$ partiendo de $\{p\} S \{p\}$. Tener en cuenta: (a) a partir de $\{p\} S \{p\}$ se puede probar $\{p \wedge p\} S \{p\}$ (por la regla de consecuencia CONS), desde donde entonces luego se puede aplicar la regla de la repetición REP. (b) La aserción false implica cualquier otra.